

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA




# Tecnologias Para o Desenvolvimento Web

**Raimundo Marcos Gonçalves Ludgério**
Professor

E-mail: raimundo.marcos@ifpb.edu.br

# Conteúdo programático

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

- Introdução ao CSS Grid;
- Fundamentos do CSS Grid;
- Terminologia importante do CSS Grid;
- Grid Line (Linha da Grade);
- Grid Track (Faixa da Grade);
- Grid Area (Área da Grade);
- Grid Item (Item da Grade);
- Grid Cell (Célula da Grade);

# CSS display

# O QUE É O DISPLAY?

A propriedade **display** do CSS define como um elemento deve ser exibido na página.

Ela controla o comportamento de **layout do elemento**.

# BLOCK (BLOCOS)

Este elemento deve ser exibido como um **bloco**.

Ele ocupa **toda a largura disponível** na tela (ou no contêiner) e começa em uma nova linha.

# BLOCK (BLOCOS)

Mesmo que o conteúdo interno seja pequeno, o elemento com **display: block** vai se estender até o final da linha (largura total do contêiner pai).

# BLOCK (BLOCOS)

Nenhum outro elemento ficará ao lado dele na mesma linha, mesmo que haja espaço.

```
<p>Exemplos de elementos com <strong>display: block</strong> por padrão:</p>
```

```
p {
    background-color: #dbd3c9;
    font-size: 1rem;
    text-align: justify;
}

strong {
    background-color: #c9c9db;
    display: block;
}
```

# BLOCK(BLOCOS)



Exemplos de elementos com
**display: block**
por padrão:

# BLOCK(BLOCOS)

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

Elementos que já estão como padrão “display:block”:

- <h1>, <h2>, <h3>, <h4>, <h5>, <h6>
- <p>
- , <section>, <article>
- <ul>, <ol>, e <li>

# O QUE É O DISPLAY?



Com display block, é possível alterar as propriedades:

- width
- height
- margin
- padding




# INLINE(EM LINHA)

Este elemento será exibido **na mesma linha** que os elementos anteriores ou seguintes, sem quebrar a linha e sem ocupar toda a largura disponível.

# INLINE(EM LINHA)

```html
<ul>
    <li>Utilizando</li>
    <li>Inline</li>
    <li>Para</li>
    <li>Colocar</li>
    <li>Lado</li>
    <li>a</li>
    <li>lado</li>
</ul>
```

```css
ul {
    background-color: palevioletred;
}

li {
    display: inline;
}
```

# INLINE(EM LINHA)



Utilizando Inline Para Colocar Lado a lado

# INLINE(EM LINHA)

É possível aplicar padding e margin em elementos inline, mas:

- padding horizontal (esquerda/direita) funciona bem
- padding vertical (topo/baixo) pode não aumentar a altura visível
- margin-top e margin-bottom em elementos inline geralmente não afetam o layout como esperado

# INLINE(EM LINHA)




Elementos que já são inline por padrão:

- 
- <a>
- , 
- <label>

# Display Grid

# CSS Grid Layout

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

Um sistema de layout baseado em **grade bidimensional** que, em comparação com qualquer sistema de layout web do passado, muda completamente a forma como projetamos interfaces de usuário.

# CSS Grid Layout

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

Primeiro usamos tabelas, depois floats, posicionamento e inline-block, mas todos esses métodos eram, essencialmente, "gambiarras" e deixavam de fora muitas funcionalidades importantes (como o alinhamento vertical, por exemplo).

# CSS Grid Layout

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

Desde março de 2017, a maioria dos navegadores passou a oferecer suporte nativo e sem prefixos ao CSS Grid: Chrome (inclusive no Android), Firefox, Safari (inclusive no iOS) e Opera.

# Grids

Para começar, definimos um elemento contêiner como uma grade

```
.container {
    display: grid;
}
```




# GRID CONTAINER

É o elemento no qual se aplica **display: grid**.

Ele é o pai direto de todos os itens da grade.

# Grids

Os tamanhos das colunas e linhas com **grid-template-columns** e **grid-template-rows**

Depois posicionar os elementos filhos dentro da grade usando **grid-column** e **grid-row**

# Grids



```
.container {
    display: grid;
    grid-template-columns: max-content;
    grid-template-rows: max-content;
    grid-column: inherit;
    grid-row: initial;
}
```

# PROPRIEDADES GRIDS

# LINHAS E COLUNAS

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

**grid-template-columns:**

- Define as colunas e linhas da grade com uma lista de valores separados por espaços.
- Os valores representam o tamanho da trilha, e o espaço entre eles representa a linha da grade.

# LINHAS E COLUNAS



```
<div class="container-2">
    <div class="item-container">Elemento A</div>
    <div class="item-container">Elemento B</div>
    <div class="item-container">Elemento C</div>
</div>
```

# LINHAS E COLUNAS



```css
.container-2 {
    border: 1px solid blue;
    display: grid;
    grid-template-columns: 1fr 5fr 1fr;
}
```

| Elemento A | Elemento B | Elemento C |
|---|---|---|

# LINHAS E COLUNAS

É possível utilizar a função “**repeat()**”do CSS:

```
grid-template-columns: repeat(4, 1fr);
```

| Elemento A | Elemento B | Elemento C | |
|---|---|---|---|

# LINHAS E COLUNAS

**grid-template-rows:**

- Define a altura das linhas de um grid
- Define quantas linhas o grid terá e a altura de cada uma

# LINHAS E COLUNAS



```
<div class="container-2">
    <div class="item-container">Elemento A</div>
    <div class="item-container">Elemento B</div>
    <div class="item-container">Elemento C</div>
    <div class="item-container">Elemento D</div>
    <div class="item-container">Elemento E</div>
    <div class="item-container">Elemento F</div>
</div>
```

# LINHAS E COLUNAS

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

```
grid-template-columns: repeat(2, 1fr);
grid-template-rows: 1fr 3fr 1fr;
```

| | |
|---|---|
| Elemento A | Elemento B |
| Elemento C | Elemento D |
| Elemento E | Elemento F |

# LINHAS E COLUNAS

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

É possível também nomear as colunas e as linhas com a sintaxe **tamanho [nome]** para cada linha/coluna.

```
grid-template-columns: 1fr [coluna-1] 1fr [coluna-2] 1fr [coluna-3] 1fr [coluna-4];
grid-template-rows: 2fr [linha-1] 2fr [linha-2] 2fr [linha-3];
```

# GRIA ÁREA



**grid-template-areas:**

- Define um modelo de grade referenciando os nomes das áreas da grade especificadas com a propriedade grid-area
- Define visualmente o layout da grade, atribuindo nomes às áreas do grid de forma intuitiva e legível.

# GRID TEMPLATE

**grid-template:**

- É um shorthand que agrupa o grid-template-rows, grid-template-columns e grid-template-areas

# GRID TEMPLATE



```
.container-3 {
    display: grid;
    grid-template:
        "header header header" 1fr
        "sidebar main . " 150px
        "footer footer footer" 1fr
        / 1fr 2fr 1fr;
}
```

# GRID TEMPLATE





# GAP

**column-gap:**

- Espaço entre as colunas

**row-gap:**

- Espaço entre as linhas

# GAP



```css
.container-3 {
    display: grid;
    grid-template:
        "header header header" 1fr
        "sidebar main ." 150px
        "footer footer footer" 1fr
        / 1fr 2fr 1fr;
    gap: 15px
}
```



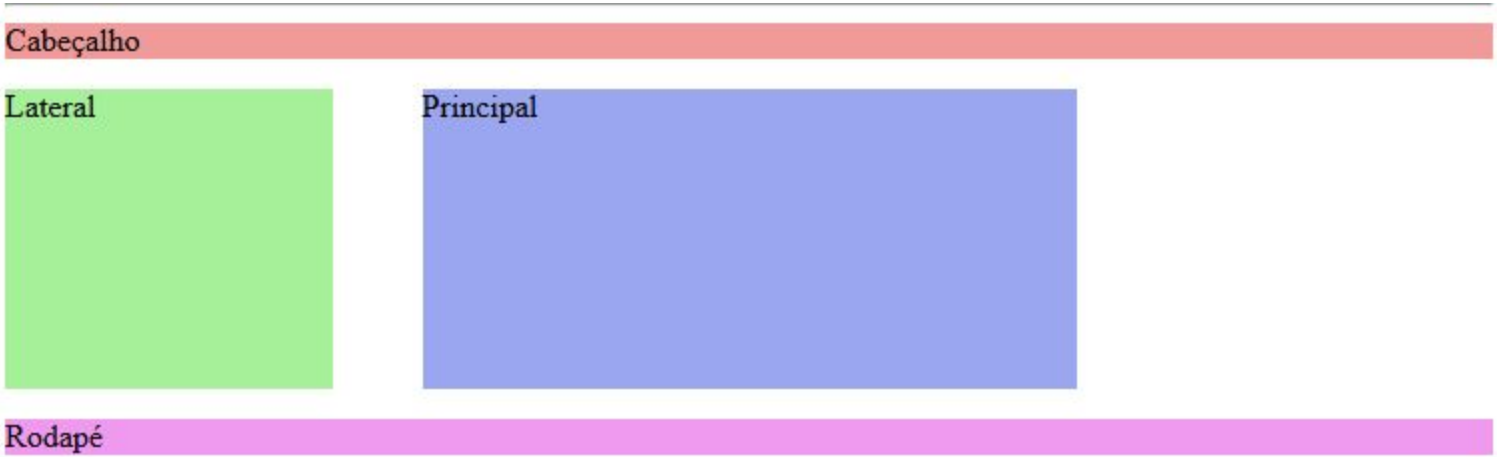
# GAP



```css
.container-3 {
    display: grid;
    grid-template:
        "header header header" 1fr
        "sidebar main ." 150px
        "footer footer footer" 1fr
        / 1fr 2fr 1fr;
    gap: 15px 45px;
}
```

# GAP





# ALINHAMENTO VERTICAL



**justify-items:**

Alinha os itens da grade ao longo do eixo inline (linha) (ao contrário de align-items, que alinha ao longo do eixo block (coluna)). Este valor se aplica a todos os itens da grade dentro do contêiner.

# ALINHAMENTO

**justify-items:**

- start: alinha os itens para que fiquem alinhados com a borda inicial da célula
- end: alinha os itens para que fiquem alinhados com a borda final da célula
- center: alinha os itens no centro da célula
- stretch: preenche toda a largura da célula (este é o padrão)

# ALINHAMENTO VERTICAL



```
<h1>Testando alinhamentos na GRID</h1>
<section class="container-direita">
    <aside>Alinhamento à direita</aside>
</section>
<section class="container-esquerda">
    <aside>Alinhamento à esquerda</aside>
</section>
<section class="container-centralizado">
    <aside>Centro</aside>
</section>
```

# ALINHAMENTO VERTICAL



```css
.container-direita {
    display: grid;
    justify-items: end;
    background-color: blanchedalmond;
}

.container-esquerda {
    display: grid;
    justify-items: start;
    background-color: rgb(205, 233, 255);
}

.container-centralizado {
    display: grid;
    justify-items: center;
    background-color: rgb(255, 205, 205);
}
```

# ALINHAMENTO VERTICAL



## Testando alinhamentos na GRID

| |
|---|
| Alinhamento à direita |
| Alinhamento à esquerda |
| Centro |

# ALINHAMENTO HORIZONTAL



**align-items:**

Alinha os itens da grade ao longo do eixo do bloco (coluna) (ao contrário de justify-items, que alinha ao longo do eixo inline (linha)).

Este valor se aplica a todos os itens da grade dentro do contêiner.

# ALINHAMENTO HORIZONTAL



**align-items:**

- start: alinha os itens para que fiquem alinhados com a borda inicial da célula
- end: alinha os itens para que fiquem alinhados com a borda final da célula
- center: alinha os itens no centro da célula
- stretch: preenche toda a altura da célula (este é o padrão)
- baseline: alinha os itens ao longo da linha de base do texto

# ALINHAMENTO VERTICAL



```
<h1>Testando alinhamentos na GRID</h1>
<section class="container-cima">
    <aside>Alinhamento em cima</aside>
</section>
<section class="container-meio">
    <aside>Alinhamento meio</aside>
</section>
<section class="container-baixo">
    <aside>Embaixo</aside>
</section>
<section class="container-base">
    <aside>baseline</aside>
</section>
```

# ALINHAMENTO VERTICAL



```css
.container-cima {
    display: grid;
    grid-template-rows: 90px;
    align-items: start;
    background-color: rgb(215, 211, 163);
}

.container-meio {
    display: grid;
    grid-template-rows: 90px;
    align-items: center;
    background-color: rgb(163, 166, 215);
}

.container-baixo {
    display: grid;
    grid-template-rows: 90px;
    align-items: end;
    background-color: rgb(215, 163, 215);
}

.container-base {
    display: grid;
    align-items: baseline;
    grid-template-rows: 90px;
    background-color: rgb(167, 215, 163);
}
```

# ALINHAMENTO VERTICAL





# ALINHAMENTO DO CONTAINER



**justify-content:**

Às vezes o tamanho total da sua grade pode ser menor que o tamanho do seu contêiner.

Esta propriedade alinhar a grade ao longo do eixo inline (linha) (ao contrário de align-content, que alinha a grade ao longo do eixo block (column)).

# JUSTIFY-CONTENT



- start: alinhar a grade para que fique nivelada com a borda inicial do contêiner da grade
- end: alinhar a grade para que fique nivelada com a borda final do contêiner da grade
- center: alinha a grade no centro do contêiner da grade
- stretch – redimensiona os itens da grade para permitir que a grade preencha toda a largura do contêiner da grade

# JUSTIFY-CONTENT




- space-around: coloca uma quantidade igual de espaço entre cada item da grade, com espaços de metade do tamanho nas extremidades
- space-between: coloca uma quantidade igual de espaço entre cada item da grade, sem espaço nas extremidades
- space-evenly: coloca uma quantidade igual de espaço entre cada item da grade, incluindo as extremidades

# JUSTIFY-CONTENT



```
<h1>Testando alinhamentos na Justify-content</h1>
<section class="container-justificado-esquerda">
    <aside>Alinhado o container à esquerda</aside>
</section>
<section class="container-justificado-direita">
    <aside>Alinhado o container à direita</aside>
</section>
<section class="container-justificado-centro">
    <aside>Alinhado o container no meio</aside>
</section>
```

# JUSTIFY-CONTENT



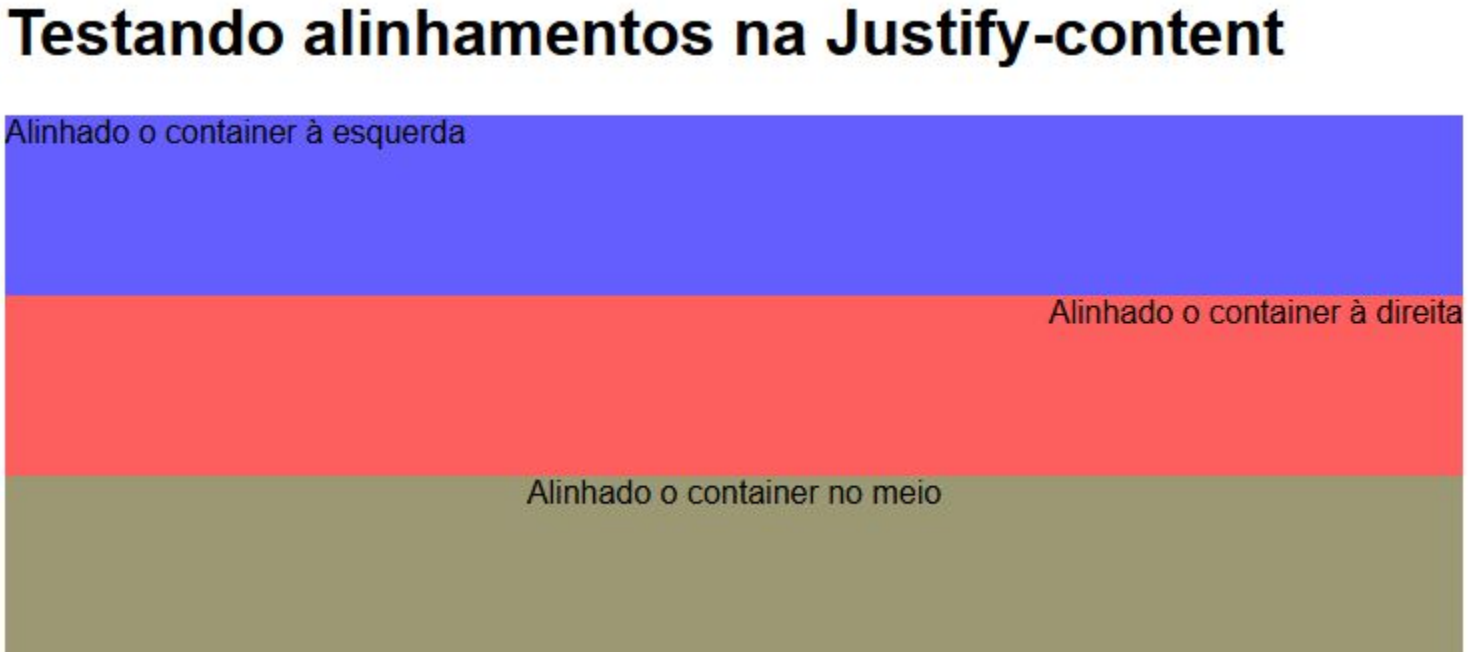
```css
.container-justificado-direita {
    display: grid;
    justify-content: end;
    background-color: rgb(253, 94, 94);
}

.container-justificado-centro {
    display: grid;
    justify-content: center;
    background-color: rgb(157, 152, 116)
}

.container-justificado-espaço-entre {
    display: grid;
    justify-content: space-between;
    background-color: rgb(116, 154, 157)
}
```

# JUSTIFY-CONTENT





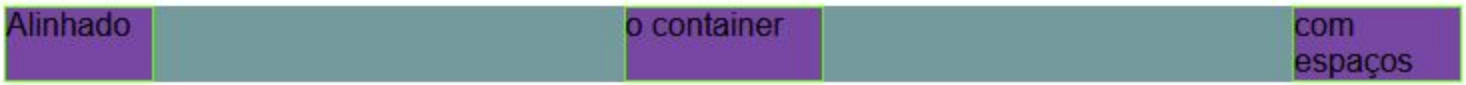
# JUSTIFY-CONTENT



```html
<section class="container-justificado-espaço-entre">
    <aside>Alinhado </aside>
    <aside>o container </aside>
    <aside>com espaços</aside>
</section>
```

```css
.container-justificado-espaço-entre {
    display: grid;
    grid-template-columns: 75px 100px 85px;
    grid-template-rows: 75px;
    justify-content: space-between;
    background-color: rgb(116, 154, 157)
}
```

# JUSTIFY-CONTENT



# JUSTIFY-CONTENT



```html
<section class="container-justificado-espaço-redor">
    <aside>Alinhado </aside>
    <aside>o container </aside>
    <aside>com espaços</aside>
    <aside>ao redor de cada elemento</aside>
</section>
```

```css
.container-justificado-espaço-redor {
    display: grid;
    grid-template-columns: 70px 90px 115px 250px;
    grid-template-rows: 70px;
    justify-content: space-around;
    background-color: rgb(213, 94, 253);
}
```

# JUSTIFY-CONTENT



# JUSTIFY-CONTENT



```html
<section class="container-justificado-espaço-redor">
    <aside>Alinhado </aside>
    <aside>o container </aside>
    <aside>com espaços</aside>
    <aside>ao redor de cada elemento</aside>
</section>
```

```css
.container-justificado-espaço-redor {
    display: grid;
    grid-template-columns: 70px 90px 115px 250px;
    grid-template-rows: 70px;
    justify-content: space-around;
    background-color: rgb(213, 94, 253);
}
```

# JUSTIFY-CONTENT



# CSS flex Layout

# Flexbox

O **display: flex** facilita o alinhamento, a distribuição e o redimensionamento de elementos dentro de um contêiner.

Ele foi criado para lidar com limitações do modelo tradicional de caixas do CSS, especialmente quando se trata de layouts responsivos.




# Flexbox

Ao aplicar **display: flex** a um elemento pai (chamado de contêiner flex), seus filhos diretos (chamados de itens flex) passam a ser organizados em um eixo principal (horizontal por padrão) e um eixo cruzado (perpendicular ao principal, ou seja, vertical por padrão).

# EIXOS DO FLEXBOX

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

- Eixo Principal:
  - Padrão: horizontal (esquerda para direita)
  - Controlado por **flex-direction**
- Eixo Cruzado:
  - Perpendicular ao eixo principal
  - Padrão: vertical (de cima para baixo)

# FLEX-DIRECTION



Define a direção do eixo principal:

- row: calor padrão da esquerda para direita em linha horizontalmente
- row-reverse: valor alinhado horizontalmente da direita para esquerda
- column: alinhado verticalmente de cima para baixo
- column-reverse: alinhado verticalmente de baixo para cima




# JUSTIFY-CONTENT

Alinha os itens ao longo do eixo principal:

```css
.container {
    display: flex;
    justify-content: flex-start;    /* padrão: alinhado ao início */
    justify-content: flex-end;      /* alinhado ao final */
    justify-content: center;        /* centralizado */
    justify-content: space-between; /* espaço igual entre os itens */
    justify-content: space-around;  /* espaço igual ao redor dos itens */
    justify-content: space-evenly;  /* espaço igual entre e ao redor */
}
```

# ALIGN-ITEMS

Alinha os itens ao longo do eixo cruzado:

```css
.container {
    display: flex;
    align-items: stretch;
    /* padrão: estica os itens para preencher */
    align-items: flex-start;
    /* topo */
    align-items: flex-end;
    /* base */
    align-items: center;
    /* centro vertical */
    align-items: baseline;
    /* alinha pelas linhas de base do texto */
}
```

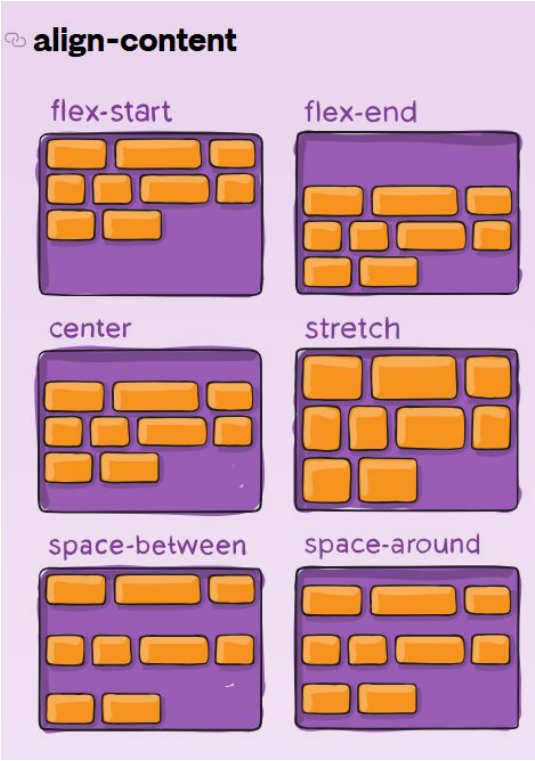



# ALIGN-CONTENT

Controla o alinhamento de várias linhas (se houver quebra):

- flex-start
- flex-end
- center
- space-between
- stretch (padrão)

# ALIGN-CONTENT

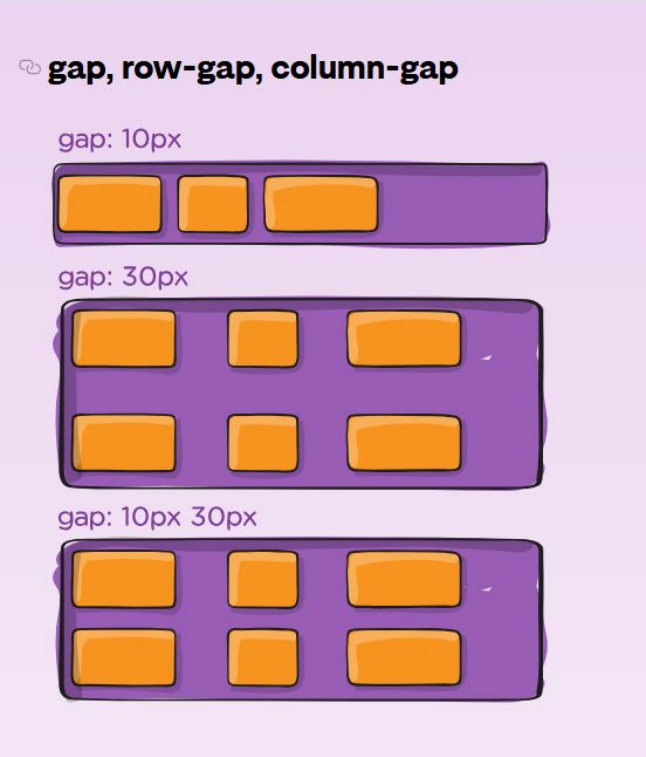


# GAP, ROW-GAP, COLUMN-GAP

A propriedade gap controla explicitamente o espaço entre os itens flexíveis.

Ela aplica esse espaçamento apenas entre os itens, não nas bordas externas.

# GAP, ROW-GAP, COLUMN-GAP



# Propriedades para os elementos filhos

Por padrão, os itens flexíveis são dispostos na ordem de origem

No entanto, a propriedade **order** controla a ordem em que eles aparecem no contêiner flexível

# JAVASCRIPT

# JavaScript (JS)

- Manipulação do DOM
- Interatividade
- Validação de formulários
- Requisições assíncronas (AJAX/fetch)
- Armazenamento local de dados
- Criação de componentes dinâmicos
- Integrações com APIs e bibliotecas

# COMO ADICIONAR JAVASCRIPT




- Utilizando a tag **<script>**
- Utilizando arquivo externo com extensão js
- No final do **<body> (melhora a performance)**

# COMO ADICIONAR JAVASCRIPT



```
<body>
    <script>
        console.log("Olá, mmundo!")
    </script>
</body>
```

# Princípios Básicos da Linguagem

# VARIÁVEIS E CONSTANTES

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

Existem três maneiras de declarar variáveis:

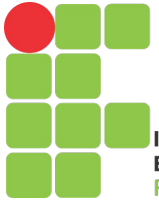
- **let:** variável de escopo
- **const:** constantes
- **var:** variáveis globais

Obs: O **;** no JavaScript é opcional.




# Tipos Primitivos

- String: Sequência de texto
- Number: Valores numéricos
- Boolean: Verdadeiro ou falso
- Array: Estrutura que permite armazenar vários valores em um único referência
- Object: Tudo em JavaScript é um objeto e pode ser armazenados em uma variável



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

# COMENTÁRIOS

- /* */ - Comentário em bloco
- // - Comentário em linha




# Operadores

Um operator é um **símbolo matemático** que produz um **resultado** baseado em dois valores (ou variáveis).

# Operadores

| Operador | Detalhes | Símbolo | Exemplo |
|---|---|---|---|
| Adição | Adicionar dois números.<br>Concatenar duas Strings | + | 5 + 6;<br>“Olá” + “Mundo”; |
| Subtração, multiplicação e divisão | Exatamente a mesma coisa da matemática | -,<br>*,<br>/ | 9 - 3;<br>8 * 2;<br>15 / 5; |
| Atribuição | Guardar um valor e associar a uma referência | = | let nome = “Raimundo”; |
| Igualdade Restrita | Verifica se dois valores são iguais. | === | let num = 3;<br>3 === 4; (falso) |
| Negação | Utilizado para negar um valor booleano | ! | !(3===4); |
| Diferente | Compara se dois valores não são iguais | !== | let nome = “a”;<br>nome !== “b”; |

# Operadores

| Operador | Detalhes | Símbolo | Exemplo |
|---|---|---|---|
| E lógico | Verdade quando as duas operações forem verdade | && | true && true |
| OU lógico | Verdade quando pelo menos uma das operações forem verdades | \|\| | false \|\| true |

# Operadores

JavaScript permite usar operadores lógicos com valores que não são booleanos.

O comportamento é baseado na ideia de valores true e false.

```
console.log(null || "valor padrão"); // "valor padrão"
console.log("texto" && 123);         // 123
console.log(0 && "oi");              // 0 (false)
console.log(!0);                     // true
```




# Condicionais

Condicionais são estruturas de código usadas para testar se uma expressão retorna verdadeira ou não.

Uma forma muito comum de condicionais é a instrução **if...else**.

# Condicionais



```
<input type="text" name="nome" id="nome" placeholder="Informe o seu nome">
<script>
    let nome = document.getElementById("nome").value;
    if (nome === "Raimundo") window.alert("Olá, Raimundo. Você é o administrdor");
    else window.alert("Verifique os dados e tente novamente! ");
</script>
```

# Switch/case

Para condições simples, o JavaScript contém a estrutura Switch/case.

```
let cor = "vermelho";
switch (cor) {
    case "azul":
        console.log("Cor do céu");
        break;
    case "vermelho":
        console.log("Cor do fogo");
        break;
    default:
        console.log("Outra cor");
}
```




# LAÇOS

## Laço for:

```
for (let i = 0; i < 5; i++) {
    console.log("Número: " + i);
}
```




# LAÇOS

## Laço while:

```
let contador = 0;
while (contador < 3) {
  console.log("Repetição " + contador);
  contador++;
}
```

# LAÇOS

## Laço do…while:

```
let i = 0;
do {
    console.log("Número:", i);
    i++;
} while (i < 5);
```

# Funções

**Functions** são uma forma de empacotar a funcionalidade que são reutilizáveis.

É possível definir um bloco de código como uma função que é executada quando é chamado pelo nome da função.

# FUNÇÕES

- **getElementById():** Retorna o elemento HTML com o ID especificado
- **getElementsByClassName():** Retorna uma coleção de elementos que possuem a classe especificada
- **getElementsByTagName():** Retorna uma coleção de todos os elementos com a tag HTML especificada (por exemplo, div, p, a)

# FUNÇÕES

- **querySelector():** Retorna o elemento no documento que corresponde ao seletor fornecido.
- **querySelectorAll():** Retorna uma NodeList (uma coleção de nós) de todos os elementos no documento que correspondem ao seletor fornecido.

# FUNÇÕES



```
// Seleciona o primeiro parágrafo com a classe .paragrafo
let primeiroParagrafo = document.querySelector('.paragrafo');
console.log(primeiroParagrafo.textContent);


// Seleciona o elemento com o id 'titulo'
let titulo = document.querySelector('#titulo');
console.log(titulo.textContent);

// Seleciona o primeiro <span>
let span = document.querySelector('span');
console.log(span.textContent);
```

# FUNÇÕES



```
let titulo = document.getElementById('titulo');
console.log(titulo.textContent);

let paragrafos = document.getElementsByClassName('paragrafo');
console.log(paragrafos[0].textContent); // Parágrafo 1
console.log(paragrafos[1].textContent); // Parágrafo 2

let spans = document.getElementsByTagName('span');
console.log(spans[0].textContent); // Texto dentro do span
```

# ARGUMENTOS

Os argumentos ficam dentro dos parênteses, separados por vírgulas se houver mais de um argumento.

```
function multiplicacao(num1, num2) {
    let result = num1 * num2;
    return result;
}
multiplicacao(8,2)
```

# ARGUMENTOS OPCIONAIS



```
function multiplicacaoOpc(num1 = 0, num2 = 1) {
    let result = num1 * num2;
    return result;
}

console.log(multiplicacaoOpc(8))
```

# VARIÁVEIS FUNÇÕES

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

O JS permite armazenar funções em variáveis

```
const minhaFun = function (numero) {
    return numero * 3
}

console.log(minhaFun(8))
```

# Funções Anônimas




Funções anônimas são funções que não possuem um nome **identificador**.

Elas são declaradas sem um nome e geralmente são usadas em contextos onde a função é usada como valor ou passada como argumento para outras funções.

# Funções Anônimas




- Não possuem nome: São definidas sem um identificador
- São expressões: Podem ser atribuídas a variáveis ou passadas como argumentos
- Úteis para callbacks: Muito usadas em eventos e operações assíncronas

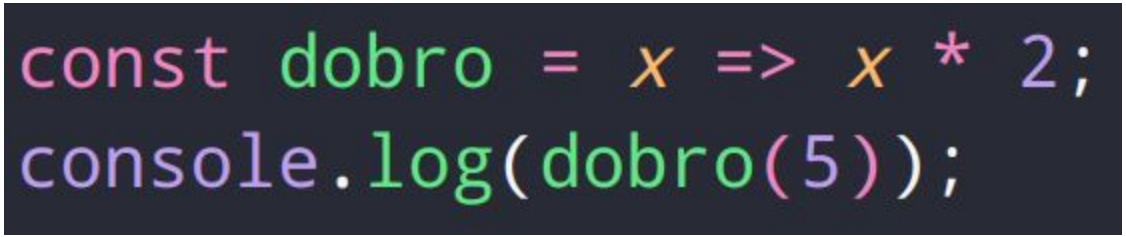
# Funções Anônimas



```
const funAnonima = () => {
    console.log("Essa é uma arrow function")
}

array.map(function(item) {
    return item * 2;
})
```

# Funções Anônimas



# Função anônima auto-executável (IIFE)



Uma IIFE (“Immediately Invoked Function Expression”, ou “Expressão de Função Invocada Imediatamente”) é um padrão em JavaScript onde você define uma função e logo em seguida a executa automaticamente — sem precisar chamá-la depois usando nome.

# OBRIGADO!

Repositório com os códigos: https://github.com/raimundoludgerio/estrutura-de-dados/tree/main/web

HOUSE, C. CSS Grid Layout Guide. 2025. Disponível em: https://css-tricks.com/snippets/css/complete-guide-grid/

COYIER, C. CSS Flexbox Layout Guide. 2024. Disponível em: https://css-tricks.com/snippets/css/a-guide-to-flexbox/#aa-flexbox-properties

Contato:

raimundo.marcos@ifpb.edu.br

As Dúvidas podem ser enviadas pelo e-mail, conversar pessoalmente após a aula ou em algum horário a combinar.